

第1章 绪论

1. 简述下列概念：数据、数据元素、数据项、数据对象、数据结构、逻辑结构、存储结构、抽象数据类型。

答案：

数据：是客观事物的符号表示，指所有能输入到计算机中并被计算机程序处理的符号的总称。如数学计算中用到的整数和实数，文本编辑所用到的字符串，多媒体程序处理的图形、图像、声音、动画等通过特殊编码定义后的数据。

数据元素：是数据的基本单位，在计算机中通常作为一个整体进行考虑和处理。在有些情况下，数据元素也称为元素、结点、记录等。数据元素用于完整地描述一个对象，如一个学生记录，树中棋盘的一个格局（状态）、图中的一个顶点等。

数据项：是组成数据元素的、有独立含义的、不可分割的最小单位。例如，学生基本信息表中的学号、姓名、性别等都是数据项。

数据对象：是性质相同的数据元素的集合，是数据的一个子集。例如：整数数据对象是集合 $N=\{0, \pm 1, \pm 2, \dots\}$ ，字母字符数据对象是集合 $C=\{ 'A', 'B', \dots, 'Z', 'a', 'b', \dots, 'z' \}$ ，学生基本信息表也可是一个数据对象。

数据结构：是相互之间存在一种或多种特定关系的数据元素的集合。换句话说，数据结构是带“结构”的数据元素的集合，“结构”就是指数据元素之间存在的关系。

逻辑结构：从逻辑关系上描述数据，它与数据的存储无关，是独立于计算机的。因此，数据的逻辑结构可以看作是从具体问题抽象出来的数学模型。

存储结构：数据对象在计算机中的存储表示，也称为**物理结构**。

抽象数据类型：由用户定义的，表示应用问题的数学模型，以及定义在这个模型上的一组操作的总称。具体包括三部分：数据对象、数据对象上关系的集合和对数据对象的基本操作的集合。

2. 试举一个数据结构的例子，叙述其逻辑结构和存储结构两方面的含义和相互关系。

答案：

例如有一张学生基本信息表，包括学生的学号、姓名、性别、籍贯、专业等。每个学生基本信息记录对应一个数据元素，学生记录按顺序号排列，形成了学生基本信息记录的线性序列。对于整个表来说，只有一个开始结点（它的前面无记录）和一个终端结点（它的后面无记录），其他的结点则各有一个也只有一个直接前趋和直接后继。学生记录之间的这种关系就确定了学生表的逻辑结构，即线性结构。

这些学生记录在计算机中的存储表示就是存储结构。如果用连续的存储单元（如用数组表示）来存放这些记录，则称为顺序存储结构；如果存储单元不连续，而是随机存放各个记录，然后用指针进行链接，则称为链式存储结构。

即相同的逻辑结构，可以对应不同的存储结构。

3. 简述逻辑结构的四种基本关系并画出它们的关系图。

答案：

（1）集合结构

数据元素之间除了“属于同一集合”的关系外，别无其他关系。例如，确定一名学生是否为班级成员，只需将班级看做一个集合结构。

（2）线性结构

数据元素之间存在一对一的关系。例如，将学生信息数据按照其入学报到的时间先后顺序进行排列，将组成一个线性结构。

（3）树结构

数据元素之间存在一对多的关系。例如，在班级的管理体系中，班长管理多个组长，每位组长管理多名组员，从而构成树形结构。

（4）图结构或网状结构

数据元素之间存在多对多的关系。例如，多位同学之间的朋友关系，任何两位同学都可以是朋友，从而构成图形结构或网状结构。

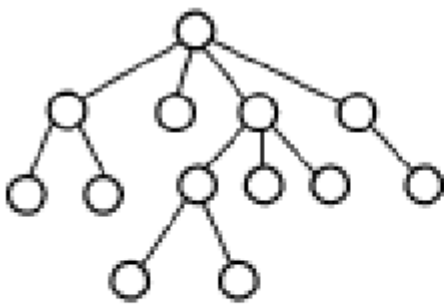
其中树结构和图结构都属于非线性结构。



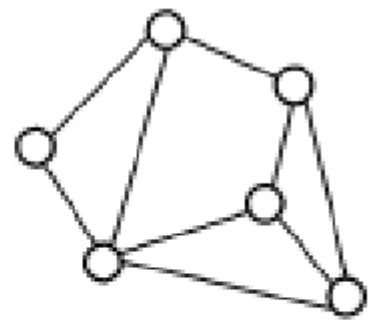
集合结构



线性结构



树结构



图结构

四类基本逻辑结构关系图

4. 存储结构由哪两种基本的存储方法实现？

答案：

（1）顺序存储结构

顺序存储结构是借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系，通常借助程序设计语言的数组类型来描述。

（2）链式存储结构

顺序存储结构要求所有的元素依次存放在一片连续的存储空间中，而链式存储结构，无需占用一整块存储空间。但为了表示结点之间的关系，需要给每个结点附加指针字段，用于存放后继元素的存储地址。所以链式存储结构通常借助于程序设计语言的指针类型来描述。

5. 选择题

（1）在数据结构中，从逻辑上可以把数据结构分成（ ）。

- A. 动态结构和静态结构
- B. 紧凑结构和非紧凑结构
- C. 线性结构和非线性结构
- D. 内部结构和外部结构

答案：C

（2）与数据元素本身的形式、内容、相对位置、个数无关的是数据的（ ）。

- A. 存储结构
- B. 存储实现
- C. 逻辑结构
- D. 运算实现

答案：C

(3) 通常要求同一逻辑结构中的所有数据元素具有相同的特性，这意味着（ ）。

- A. 数据具有同一特点
- B. 不仅数据元素所包含的数据项的个数要相同，而且对应数据项的类型要一致
- C. 每个数据元素都一样
- D. 数据元素所包含的数据项的个数要相等

答案：B

(4) 以下说法正确的是（ ）。

- A. 数据元素是数据的最小单位
- B. 数据项是数据的基本单位
- C. 数据结构是带有结构的各数据项的集合
- D. 一些表面上很不相同的数据可以有相同的逻辑结构

答案：D

解释：数据元素是数据的基本单位，数据项是数据的最小单位，数据结构是带有结构的各数据元素的集合。

(5) 算法的时间复杂度取决于（ ）。

- A. 问题的规模
- B. 待处理数据的初态
- C. 计算机的配置
- D. A 和 B

答案：D

解释：算法的时间复杂度不仅与问题的规模有关，还与问题的其他因素有关。如某些排序的算法，其执行时间与待排序记录的初始状态有关。为此，有时会对算法有最好、最坏以及平均时间复杂度的评价。

(6) 以下数据结构中，（ ）是非线性数据结构

- A. 树
- B. 字符串
- C. 队列
- D. 栈

答案：A

6. 试分析下面各程序段的时间复杂度。

```
(1) x=90; y=100;
while(y>0)
    if(x>100)
        {x=x-10; y--;}
    else x++;
```

答案：O(1)

解释：程序的执行次数为常数阶。

```
(2) for (i=0; i<n; i++)
    for (j=0; j<m; j++)
        a[i][j]=0;
```

答案：O(m*n)

解释：语句 `a[i][j]=0;` 的执行次数为 $m*n$ 。

```
(3) s=0;
for i=0; i<n; i++)
    for(j=0; j<n; j++)
        s+=B[i][j];
sum=s;
```

答案：O(n^2)

解释：语句 `s+=B[i][j];` 的执行次数为 n^2 。

```
(4) i=1;
while(i<=n)
```

```
i=i*3;
```

答案: $O(\log_3 n)$

解释: 语句 $i=i*3$; 的执行次数为 $\lfloor \log_3 n \rfloor$ 。

(5) $x=0$;

```
for(i=1; i<n; i++)
```

```
for (j=1; j<=n-i; j++)
```

```
    x++;
```

答案: $O(n^2)$

解释: 语句 $x++$; 的执行次数为 $n-1+n-2+\cdots+1 = n(n-1)/2$ 。

(6) $x=n$; $//n>1$

```
    y=0;
```

```
    while( $x \geq (y+1) * (y+1)$ )
```

```
        y++;
```

答案: $O(\sqrt{n})$

解释: 语句 $y++$; 的执行次数为 $\lfloor \sqrt{n} \rfloor$ 。

第2章 线性表

1. 选择题

(1) 顺序表中第一个元素的存储地址是 100, 每个元素的长度为 2, 则第 5 个元素的地址是()。

A. 110

B. 108

C. 100

D. 120

答案: B

解释: 顺序表中的数据连续存储, 所以第 5 个元素的地址为: $100+2*4=108$ 。

(2) 在 n 个结点的顺序表中, 算法的时间复杂度是 $O(1)$ 的操作是()。

A. 访问第 i 个结点 ($1 \leq i \leq n$) 和求第 i 个结点的直接前驱 ($2 \leq i \leq n$)

B. 在第 i 个结点后插入一个新结点 ($1 \leq i \leq n$)

C. 删除第 i 个结点 ($1 \leq i \leq n$)

D. 将 n 个结点从小到大排序

答案: A

解释: 在顺序表中插入一个结点的时间复杂度都是 $O(n^2)$, 排序的时间复杂度为 $O(n^2)$ 或 $O(n \log_2 n)$ 。

顺序表是一种随机存取结构, 访问第 i 个结点和求第 i 个结点的直接前驱都可以直接通过数组的下标直接定位, 时间复杂度是 $O(1)$ 。

(3) 向一个有 127 个元素的顺序表中插入一个新元素并保持原来顺序不变, 平均要移动__的元素个数为()。

A. 8

B. 63.5

C. 63

D. 7

答案: B

解释: 平均要移动的元素个数为: $n/2$ 。

(4) 链接存储的存储结构所占存储空间()。

A. 分两部分, 一部分存放结点值, 另一部分存放表示结点间关系的指针

B. 只有一部分, 存放结点值

C. 只有一部分, 存储表示结点间关系的指针

D. 分两部分, 一部分存放结点值, 另一部分存放结点所占单元数

答案: A

(5) 线性表若采用链式存储结构时, 要求内存中可用存储单元的地址()。

- A. 必须是连续的 B. 部分地址必须是连续的
C. 一定是不连续的 D. 连续或不连续都可以

答案: D

(6) 线性表 L 在 () 情况下适用于使用链式结构实现。

- A. 需经常修改 L 中的结点值 B. 需不断对 L 进行删除插入
C. L 中含有大量的结点 D. L 中结点结构复杂

答案: B

解释: 链表最大的优点在于插入和删除时不需要移动数据, 直接修改指针即可。

(7) 单链表的存储密度 ()。

- A. 大于 1 B. 等于 1 C. 小于 1 D. 不能确定

答案: C

解释: 存储密度是指一个结点数据本身所占的存储空间和整个结点所占的存储空间之比, 假设单链表一个结点本身所占的空间为 D, 指针域所占的空间为 N, 则存储密度为: $D/(D+N)$, 一定小于 1。

(8) 将两个各有 n 个元素的有序表归并成一个有序表, 其最少的比较次数是 ()。

- A. n B. $2n-1$ C. $2n$ D. $n-1$

答案: A

解释: 当第一个有序表中所有的元素都小于 (或大于) 第二个表中的元素, 只需要用第二个表中的第一个元素依次与第一个表的元素比较, 总计比较 n 次。

(9) 在一个长度为 n 的顺序表中, 在第 i 个元素 ($1 \leq i \leq n+1$) 之前插入一个新元素时须向后移动 () 个元素。

- A. $n-i$ B. $n-i+1$ C. $n-i-1$ D. 1

答案: B

(10) 线性表 $L=(a_1, a_2, \dots, a_n)$, 下列说法正确的是 ()。

- A. 每个元素都有一个直接前驱和一个直接后继
B. 线性表中至少有一个元素
C. 表中诸元素的排列必须是由小到大或由大到小
D. 除第一个和最后一个元素外, 其余每个元素都有一个且仅有一个直接前驱和直接后继。

答案: D

(11) 创建一个包括 n 个结点的有序单链表的时间复杂度是 ()。

- A. $O(1)$ B. $O(n)$ C. $O(n^2)$ D. $O(n \log_2 n)$

答案: C

解释: 单链表创建的时间复杂度是 $O(n)$, 而要建立一个有序的单链表, 则每生成一个新结点时需要和已有的结点进行比较, 确定合适的插入位置, 所以时间复杂度是 $O(n^2)$ 。

(12) 以下说法错误的是 ()。

- A. 求表长、定位这两种运算在采用顺序存储结构时实现的效率不比采用链式存储结构时实现的效率低
B. 顺序存储的线性表可以随机存取
C. 由于顺序存储要求连续的存储区域, 所以在存储管理上不够灵活
D. 线性表的链式存储结构优于顺序存储结构

答案: D

解释: 链式存储结构和顺序存储结构各有优缺点, 有不同的适用场合。

(13) 在单链表中, 要将 s 所指结点插入到 p 所指结点之后, 其语句应为 ()。

- A. $s \rightarrow \text{next} = p+1$; $p \rightarrow \text{next} = s$;
B. $(*p).\text{next} = s$; $(*s).\text{next} = (*p).\text{next}$;
C. $s \rightarrow \text{next} = p \rightarrow \text{next}$; $p \rightarrow \text{next} = s \rightarrow \text{next}$;

D. $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$

答案: D

(14) 在双向链表存储结构中, 删除 p 所指的结点时须修改指针 ()。

A. $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}; p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next};$

B. $p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next}; p \rightarrow \text{next} \rightarrow \text{prior} = p;$

C. $p \rightarrow \text{prior} \rightarrow \text{next} = p; p \rightarrow \text{prior} = p \rightarrow \text{prior} \rightarrow \text{prior};$

D. $p \rightarrow \text{prior} = p \rightarrow \text{next} \rightarrow \text{next}; p \rightarrow \text{next} = p \rightarrow \text{prior} \rightarrow \text{prior};$

答案: A

(15) 在双向循环链表中, 在 p 指针所指的结点后插入 q 所指向的新结点, 其修改指针的操作是 ()

A. $p \rightarrow \text{next} = q; q \rightarrow \text{prior} = p; p \rightarrow \text{next} \rightarrow \text{prior} = q; q \rightarrow \text{next} = q;$

B. $p \rightarrow \text{next} = q; p \rightarrow \text{next} \rightarrow \text{prior} = q; q \rightarrow \text{prior} = p; q \rightarrow \text{next} = p \rightarrow \text{next};$

C. $q \rightarrow \text{prior} = p; q \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} \rightarrow \text{prior} = q; p \rightarrow \text{next} = q;$

D. $q \rightarrow \text{prior} = p; q \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = q; p \rightarrow \text{next} \rightarrow \text{prior} = q;$

答案: C

第3章 栈和队列

1. 选择题

(1) 若让元素 1, 2, 3, 4, 5 依次进栈, 则出栈次序不可能出现在 () 种情况。

A. 5, 4, 3, 2, 1 B. 2, 1, 5, 4, 3 C. 4, 3, 1, 2, 5

D. 2, 3, 5, 4, 1

答案: C

解释: 栈是后进先出的线性表, 不难发现 C 选项中元素 1 比元素 2 先出栈, 违背了栈的后进先出原则, 所以不可能出现 C 选项所示的情况。

(2) 若已知一个栈的入栈序列是 1, 2, 3, ..., n , 其输出序列为 $p_1, p_2, p_3, \dots, p_n$, 若 $p_1 = n$, 则 p_i 为 ()。

A. i

B. $n-i$

C. $n-i+1$

D. 不确定

答案: C

解释: 栈是后进先出的线性表, 一个栈的入栈序列是 1, 2, 3, ..., n , 而输出序列的第一个元素为 n , 说明 1, 2, 3, ..., n 一次性全部进栈, 再进行输出, 所以 $p_1 = n, p_2 = n-1, \dots, p_i = n-i+1$ 。

(3) 数组 $Q[n]$ 用来表示一个循环队列, f 为当前队列头元素的前一位置, r 为队尾元素的位置, 假定队列中元素的个数小于 n , 计算队列中元素个数的公式为 ()。

A. $r-f$

B. $(n+f-r)\%n$

C. $n+r-f$

D. $(n+r-f)\%n$

答案: D

解释: 对于非循环队列, 尾指针和头指针的差值便是队列的长度, 而对于循环队列, 差值可能为负数, 所以需要将差值加上 MAXSIZE (本题为 n), 然后与 MAXSIZE (本题为 n) 求余, 即 $(n+r-f)\%n$ 。

(4) 链式栈结点为: (data, link), top 指向栈顶。若想摘除栈顶结点, 并将删除结点的值保存到 x 中, 则应执行操作 ()。

A. $x = \text{top} \rightarrow \text{data}; \text{top} = \text{top} \rightarrow \text{link};$

B. $\text{top} = \text{top} \rightarrow \text{link}; x = \text{top} \rightarrow \text{link};$

C. $x = \text{top}; \text{top} = \text{top} \rightarrow \text{link};$

D. $x = \text{top} \rightarrow \text{link};$

答案: A

解释: $x = \text{top} \rightarrow \text{data}$ 将结点的值保存到 x 中, $\text{top} = \text{top} \rightarrow \text{link}$ 栈顶指针指向栈顶下一结点, 即摘除栈顶结点。

(5) 设有一个递归算法如下

```
int fact(int n) { //n 大于等于 0
    if(n <= 0) return 1;
```

```
else return n*fact(n-1); }
```

则计算 $\text{fact}(n)$ 需要调用该函数的次数为 ()。

A. $n+1$

B. $n-1$

C. n

D.

$n+2$

答案: A

解释: 特殊值法。设 $n=0$, 易知仅调用一次 $\text{fact}(n)$ 函数, 故选 A。

(6) 栈在 () 中有所应用。

A. 递归调用

B. 函数调用

C. 表达式求值

D. 前三个选项都有

答案: D

解释: 递归调用、函数调用、表达式求值均用到了栈的后进先出性质。

(7) 为解决计算机主机与打印机间速度不匹配问题, 通常设一个打印数据缓冲区。主机将要输出的数据依次写入该缓冲区, 而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是 ()。

A. 队列

B. 栈

C. 线性表

D. 有序表

答案: A

解释: 解决缓冲区问题应利用一种先进先出的线性表, 而队列正是一种先进先出的线性表。

(8) 设栈 S 和队列 Q 的初始状态为空, 元素 e_1 、 e_2 、 e_3 、 e_4 、 e_5 和 e_6 依次进入栈 S , 一个元素出栈后即进入 Q , 若 6 个元素出队的序列是 e_2 、 e_4 、 e_3 、 e_6 、 e_5 和 e_1 , 则栈 S 的容量至少应该是 ()。

A. 2

B. 3

C. 4

D. 6

答案: B

解释: 元素出队的序列是 e_2 、 e_4 、 e_3 、 e_6 、 e_5 和 e_1 , 可知元素入队的序列是 e_2 、 e_4 、 e_3 、 e_6 、 e_5 和 e_1 , 即元素出栈的序列也是 e_2 、 e_4 、 e_3 、 e_6 、 e_5 和 e_1 , 而元素 e_1 、 e_2 、 e_3 、 e_4 、 e_5 和 e_6 依次进入栈, 易知栈 S 中最多同时存在 3 个元素, 故栈 S 的容量至少为 3。

(9) 若一个栈以向量 $V[1..n]$ 存储, 初始栈顶指针 top 设为 $n+1$, 则元素 x 进栈的正确操作是 ()。

A. $\text{top}++$; $V[\text{top}]=x$;

B. $V[\text{top}]=x$; $\text{top}++$;

C. $\text{top}--$; $V[\text{top}]=x$;

D. $V[\text{top}]=x$; $\text{top}--$;

答案: C

解释: 初始栈顶指针 top 为 $n+1$, 说明元素从数组向量的高端地址进栈, 又因为元素存储在向量空间 $V[1..n]$ 中, 所以进栈时 top 指针先下移变为 n , 之后将元素 x 存储在 $V[n]$ 。

(10) 设计一个判别表达式中左、右括号是否配对出现的算法, 采用 () 数据结构最佳。

A. 线性表的顺序存储结构

B. 队列

C. 线性表的链式存储结构

D. 栈

答案: D

解释: 利用栈的后进先出原则。

(11) 用链接方式存储的队列, 在进行删除运算时 ()。

A. 仅修改头指针

B. 仅修改尾指针

C. 头、尾指针都要修改

D. 头、尾指针可能都要修改

答案: D

解释: 一般情况下只修改头指针, 但是, 当删除的是队列中最后一个元素时, 队尾指针也丢失了, 因此需对队尾指针重新赋值。

(12) 循环队列存储在数组 $A[0..m]$ 中, 则入队时的操作为 ()。

A. $\text{rear}=\text{rear}+1$

B. $\text{rear}=(\text{rear}+1)\%(m-1)$

C. $\text{rear}=(\text{rear}+1)\%m$

D. $\text{rear}=(\text{rear}+1)\%(m+1)$

答案: D

解释: 数组 $A[0..m]$ 中共含有 $m+1$ 个元素, 故在求模运算时应除以 $m+1$ 。

(13) 最大容量为 n 的循环队列, 队尾指针是 rear , 队头是 front , 则队空的条件是 ()。

- A. $(rear+1)\%n==front$ B. $rear==front$
C. $rear+1==front$ D. $(rear-1)\%n==front$

答案: B

解释: 最大容量为 n 的循环队列, 队满条件是 $(rear+1)\%n==front$, 队空条件是 $rear==front$ 。

(14) 栈和队列的共同点是 ()。

- A. 都是先进先出 B. 都是先进后出
C. 只允许在端点处插入和删除元素 D. 没有共同点

答案: C

解释: 栈只允许在栈顶处进行插入和删除元素, 队列只允许在队尾插入元素和在队头删除元素。

(15) 一个递归算法必须包括 ()。

- A. 递归部分 B. 终止条件和递归部分
C. 迭代部分 D. 终止条件和迭代部分

答案: B

第4章 串、数组和广义表

1. 选择题

(1) 串是一种特殊的线性表, 其特殊性体现在 ()。

- A. 可以顺序存储 B. 数据元素是一个字符
C. 可以链式存储 D. 数据元素可以是多个字符若

答案: B

(2) 串下面关于串的的叙述中, () 是不正确的?

- A. 串是字符的有限序列 B. 空串是由空格构成的串
C. 模式匹配是串的一种重要运算 D. 串既可以采用顺序存储, 也可以采用链式存储

答案: B

解释: 空格常常是串的字符集合中的一个元素, 有一个或多个空格组成的串成为空格串, 零个字符的串成为空串, 其长度为零。

(3) 串 “ababaaababaa” 的 next 数组为 ()。

- A. 012345678999 B. 0121211111212 C. 011234223456 D. 0123012322345

答案: C

(4) 串 “ababaabab” 的 nextval 为 ()。

- A. 010104101 B. 010102101 C. 010100011 D. 010101011

答案: A

(5) 串的长度是指 ()。

- A. 串中所含不同字母的个数 B. 串中所含字符的个数
C. 串中所含不同字符的个数 D. 串中所含非空格字符的个数

答案: B

解释: 串中字符的数目称为串的长度。

(6) 假设以行序为主序存储二维数组 $A=array[1..100,1..100]$, 设每个数据元素占 2 个存储单元, 基地址为 10, 则 $LOC[5,5]=$ ()。

- A. 808 B. 818 C. 1010 D. 1020

答案: B

解释: 以行序为主, 则 $LOC[5,5]=[(5-1) * 100 + (5-1)] * 2 + 10 = 818$ 。

(7) 设有数组 $A[i,j]$, 数组的每个元素长度为 3 字节, i 的值为 1 到 8, j 的值为 1 到 10, 数组从内存首地址 BA 开始顺序存放, 当用以列为序存放时, 元素 $A[5,8]$ 的存储首地址为 ()。

- A. $BA+141$ B. $BA+180$ C. $BA+222$ D. $BA+225$

答案: B

解释: 以列序为主, 则 $LOC[5,8]=[(8-1) * 8 + (5-1)] * 3 + BA = BA + 180$ 。

(8) 设有一个 10 阶的对称矩阵 A , 采用压缩存储方式, 以行序为主存储, a_{11} 为第一元素, 其存储地址为 1, 每个元素占一个地址空间, 则 a_{85} 的地址为 ()。

- A. 13 B. 32 C. 33 D. 40

答案: C

(9) 若对 n 阶对称矩阵 A 以行序为主序方式将其下三角形的元素(包括主对角线上所有元素)依次存放于一维数组 $B[1..(n(n+1))/2]$ 中, 则在 B 中确定 a_{ij} ($i < j$) 的位置 k 的关系为 ()。

- A. $i*(i-1)/2+j$ B. $j*(j-1)/2+i$ C. $i*(i+1)/2+j$ D. $j*(j+1)/2+i$

答案: B

(10) 二维数组 A 的每个元素是由 10 个字符组成的串, 其行下标 $i=0, 1, \dots, 8$, 列下标 $j=1, 2, \dots, 10$ 。若 A 按行先存储, 元素 $A[8, 5]$ 的起始地址与当 A 按列先存储时的元素 () 的起始地址相同。设每个字符占一个字节。

- A. $A[8,5]$ B. $A[3,10]$ C. $A[5,8]$ D. $A[0,9]$

答案: B

解释: 设数组从内存首地址 M 开始顺序存放, 若数组按行先存储, 元素 $A[8, 5]$ 的起始地址为: $M + [(8-0) * 10 + (5-1)] * 1 = M + 84$; 若数组按列先存储, 易计算出元素 $A[3, 10]$ 的起始地址为: $M + [(10-1) * 9 + (3-0)] * 1 = M + 84$ 。故选 B。

(11) 设二维数组 $A[1..m, 1..n]$ (即 m 行 n 列) 按行存储在数组 $B[1..m*n]$ 中, 则二维数组元素 $A[i,j]$ 在一维数组 B 中的下标为 ()。

- A. $(i-1)*n+j$ B. $(i-1)*n+j-1$ C. $i*(j-1)$ D. $j*m+i-1$

答案: A

解释: 特殊值法。取 $i=j=1$, 易知 $A[1,1]$ 的下标为 1, 四个选项中仅有 A 选项能确定的值为 1, 故选 A。

(12) 数组 $A[0..4, -1..-3, 5..7]$ 中含有元素的个数 ()。

- A. 55 B. 45 C. 36 D. 16

答案: B

解释: 共有 $5 * 3 * 3 = 45$ 个元素。

(13) 广义表 $A=(a,b,(c,d),(e,(f,g)))$, 则 $Head(Tail(Head(Tail(Tail(A)))))$ 的值为 ()。

- A. (g) B. (d) C. c D. d

答案: D

解释: $Tail(A)=(b,(c,d),(e,(f,g)))$; $Tail(Tail(A))=((c,d),(e,(f,g)))$; $Head(Tail(Tail(A)))= (c,d)$; $Tail(Head(Tail(Tail(A))))=(d)$; $Head(Tail(Head(Tail(Tail(A))))))=d$ 。

(14) 广义表 $((a,b,c,d))$ 的表头是 (), 表尾是 ()。

- A. a B. () C. (a,b,c,d) D. (b,c,d)

答案: C、B

解释: 表头为非空广义表的第一个元素, 可以是一个单原子, 也可以是一个子表, $((a,b,c,d))$ 的表头为一个子表 (a,b,c,d) ; 表尾为除去表头之外, 由其余元素构成的表, 表为一定是个广义表, $((a,b,c,d))$ 的表尾为空表 $()$ 。

(15) 设广义表 $L=((a,b,c))$, 则 L 的长度和深度分别为 ()。

- A. 1 和 1 B. 1 和 3 C. 1 和 2 D. 2 和 3

答案: C

解释：广义表的深度是指广义表中展开后所含括号的层数，广义表的长度是指广义表中所含元素的个数。根据定义易知 L 的长度为 1，深度为 2。

2. 应用题

(1) 已知模式串 $t = \text{'abcaabbabcb'}$ 写出用 KMP 法求得的每个字符对应的 next 和 nextval 函数值。

答案：

模式串 t 的 next 和 nextval 值如下：

j	1 2 3 4 5 6 7 8 9 10 11 12
t 串	a b c a a b b a b c a b
next[j]	0 1 1 1 2 2 3 1 2 3 4 5
nextval[j]	0 1 1 0 2 1 3 0 1 1 0 5

(2) 设目标为 $t = \text{"abcaabbabcbacbaacbacba"}$ ，模式为 $p = \text{"abcabaa"}$

① 计算模式 p 的 nextval 函数值；

② 不写出算法，只画出利用 KMP 算法进行模式匹配时每一趟的匹配过程。

答案：

① p 的 nextval 函数值为 0110132。（ p 的 next 函数值为 0111232）。

② 利用 KMP (改进的 nextval) 算法，每趟匹配过程如下：

第一趟匹配： abcaabbabcbacbaacbacba
 abcab (i=5, j=5)

第二趟匹配： abcaabbabcbacbaacbacba
 abc (i=7, j=3)

第三趟匹配： abcaabbabcbacbaacbacba
 a (i=7, j=1)

第四趟匹配： abcaabbabcbacbaac bacba
(成功) abcabaa (i=15, j=8)

(3) 数组 A 中，每个元素 $A[i, j]$ 的长度均为 32 个二进位，行下标从 -1 到 9，列下标从 1 到 11，从首地址 S 开始连续存放主存储器中，主存储器字长为 16 位。求：

① 存放该数组所需多少单元？

② 存放数组第 4 列所有元素至少需多少单元？

③ 数组按行存放时，元素 $A[7, 4]$ 的起始地址是多少？

④ 数组按列存放时，元素 $A[4, 7]$ 的起始地址是多少？

答案：

每个元素 32 个二进制位，主存字长 16 位，故每个元素占 2 个字长，行下标可平移至 1 到 11。

(1) 242 (2) 22 (3) $s+182$ (4) $s+142$

(4) 请将香蕉 banana 用工具 $H()$ — $Head()$ ， $T()$ — $Tail()$ 从 L 中取出。

$L = (\text{apple}, (\text{orange}, (\text{strawberry}, (\text{banana})), \text{peach}), \text{pear})$

答案： $H(H(T(H(T(H(T(L)))))))$

第 5 章 树和二叉树

1. 选择题

(9) 在下列存储形式中, () 不是树的存储形式?

- A. 双亲表示法 B. 孩子链表表示法 C. 孩子兄弟表示法 D. 顺序存储表示法

答案: D

解释: 树的存储结构有三种: 双亲表示法、孩子表示法、孩子兄弟表示法, 其中孩子兄弟表示法是常用的表示法, 任意一棵树都能通过孩子兄弟表示法转换为二叉树进行存储。

(10) 一棵非空的二叉树的先序遍历序列与后序遍历序列正好相反, 则该二叉树一定满足 ()。

- A. 所有的结点均无左孩子 B. 所有的结点均无右孩子
C. 只有一个叶子结点 D. 是任意一棵二叉树

答案: C

解释: 因为先序遍历结果是“中左右”, 后序遍历结果是“左右中”, 当没有左子树时, 就是“中右”和“右中”; 当没有右子树时, 就是“中左”和“左中”。则所有的结点均无左孩子或所有的结点均无右孩子均可, 所以 A、B 不能选, 又所有的结点均无左孩子与所有的结点均无右孩子时, 均只有一个叶子结点, 故选 C。

(11) 设哈夫曼树中有 199 个结点, 则该哈夫曼树中有 () 个叶子结点。

- A. 99 B. 100 C. 101 D. 102

答案: B

解释: 在哈夫曼树中没有度为 1 的结点, 只有度为 0 (叶子结点) 和度为 2 的结点。设叶子结点的个数为 n_0 , 度为 2 的结点的个数为 n_2 , 由二叉树的性质 $n_0 = n_2 + 1$, 则总结点数 $n = n_0 + n_2 = 2 * n_0 - 1$, 得到 $n_0 = 100$ 。

(12) 若 X 是二叉中序线索树中一个有左孩子的结点, 且 X 不为根, 则 X 的前驱为 ()。

- A. X 的双亲 B. X 的右子树中最左的结点
C. X 的左子树中最右结点 D. X 的左子树中最右叶结点

答案: C

(13) 引入二叉线索树的目的是 ()。

- A. 加快查找结点的前驱或后继的速度 B. 为了能在二叉树中方便的进行插入与删除
C. 为了能方便的找到双亲 D. 使二叉树的遍历结果唯一

答案: A

(14) 设 F 是一个森林, B 是由 F 变换得的二叉树。若 F 中有 n 个非终端结点, 则 B 中右指针域为空的结点有 () 个。

- A. $n-1$ B. n C. $n + 1$ D. $n + 2$

答案: C

(15) n ($n \geq 2$) 个权值均不相同的字符构成哈夫曼树, 关于该树的叙述中, 错误的是 ()。

- A. 该树一定是一棵完全二叉树
B. 树中一定没有度为 1 的结点
C. 树中两个权值最小的结点一定是兄弟结点
D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值

答案: A

解释: 哈夫曼树的构造过程是每次都选取权值最小的树作为左右子树构造一棵新的二叉树, 所以树中一定没有度为 1 的结点、两个权值最小的结点一定是兄弟结点、任一非叶结点的权值一定不小于下一层任一结点的权值。

2. 应用题

(1) 试找出满足下列条件的二叉树

- ① 先序序列与后序序列相同 ② 中序序列与后序序列相同

- ③ 先序序列与中序序列相同 ④中序序列与层次遍历序列相同

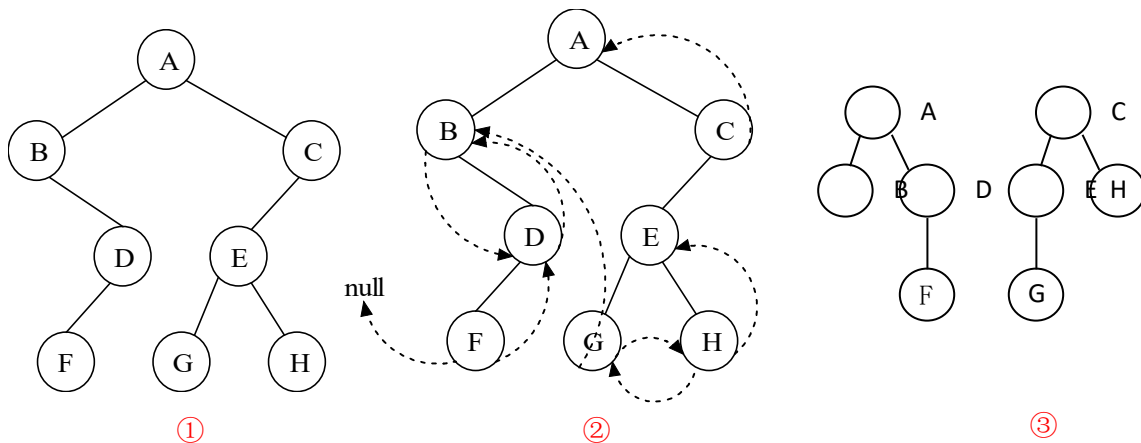
答案: 先序遍历二叉树的顺序是“根—左子树—右子树”，中序遍历“左子树—根—右子树”，后序遍历顺序是：“左子树—右子树—根”，根据以上原则有

- ① 或为空树，或为只有根结点的二叉树
- ② 或为空树，或为任一结点至多只有左子树的二叉树.
- ③ 或为空树，或为任一结点至多只有右子树的二叉树.
- ④ 或为空树，或为任一结点至多只有右子树的二叉树

(2) 设一棵二叉树的先序序列: A B D F C E G H , 中序序列: B F D A G E H C

- ①画出这棵二叉树。
- ②画出这棵二叉树的后序线索树。
- ③将这棵二叉树转换成对应的树（或森林）。

答案:



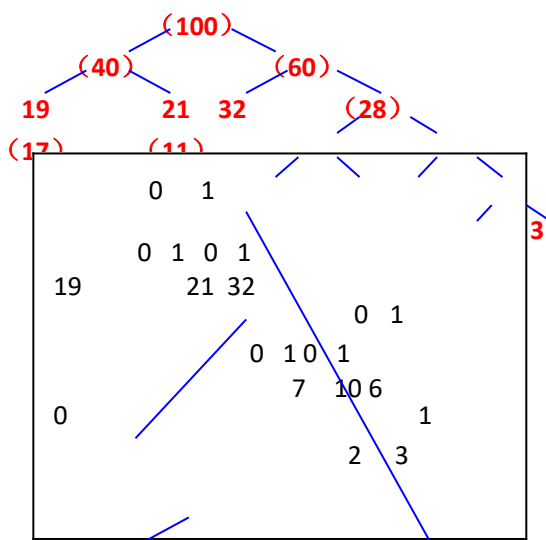
(3) 假设用于通信的电文仅由 8 个字母组成，字母在电文中出现的频率分别为 0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10。

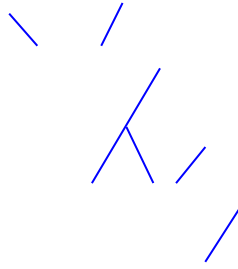
- ① 试为这 8 个字母设计赫夫曼编码。
- ② 试设计另一种由二进制表示的等长编码方案。
- ③ 对于上述实例，比较两种方案的优缺点。

答案: 方案 1: 哈夫曼编码

先将概率放大 100 倍，以方便构造哈夫曼树。

$w=\{7,19,2,6,32,3,21,10\}$ ，按哈夫曼规则: $[(2,3), 6], (7,10), \dots, 19, 21, 32$





方案比较：

字母 编号	对 应 编码	出 现 频率
1	1100	0.07
2	00	0.19
3	11110	0.02
4	1110	0.06
5	10	0.32
6	11111	0.03
7	01	0.21
8	1101	0.10

字母 编号	对应 编码	出 现 频率
1	000	0.07
2	001	0.19
3	010	0.02
4	011	0.06
5	100	0.32
6	101	0.03
7	110	0.21
8	111	0.10

方案 1 的 $WPL = 2(0.19+0.32+0.21)+4(0.07+0.06+0.10)+5(0.02+0.03)=1.44+0.92+0.25=2.61$

方案 2 的 $WPL = 3(0.19+0.32+0.21+0.07+0.06+0.10+0.02+0.03)=3$

结论：哈夫曼编码优于等长二进制编码

(4) 已知下列字符 A、B、C、D、E、F、G 的权值分别为 3、12、7、4、2、8，11，试填写出其对应哈夫曼树 HT 的存储结构的初态和终态。

答案：

初态：

	weight	parent	lchild	rchild
1	3	0	0	0
2	12	0	0	0
3	7	0	0	0
4	4	0	0	0
5	2	0	0	0
6	8	0	0	0
7	11	0	0	0
8		0	0	0
9		0	0	0
10		0	0	0
11		0	0	0
12		0	0	0
13		0	0	0

终态:

	weight	parent	lchild	rchild
1	3	8	0	0
2	12	12	0	0
3	7	10	0	0
4	4	9	0	0
5	2	8	0	0
6	8	10	0	0
7	11	11	0	0
8	5	9	5	1
9	9	11	4	8
10	15	12	3	6
11	20	13	9	7
12	27	13	2	10
13	47	0	11	12

第6

章图

1. 选择
(1)
度数之和

题
在一个图中，所有顶点的
等于图的边数的()倍。

- A. 1/2 B. 1 C. 2 D. 4

答案: C

(2) 在一个有向图中，所有顶点的入度之和等于所有顶点的出度之和的()倍。

- A. 1/2 B. 1 C. 2 D. 4

答案: B

解释: 有向图所有顶点入度之和等于所有顶点出度之和。

(3) 具有n个顶点的有向图最多有()条边。

- A. n B. n(n-1) C. n(n+1) D. n²

答案: B

解释: 有向图的边有方向之分，即为从n个顶点中选取2个顶点有序排列，结果为n(n-1)。

(4) n个顶点的连通图用邻接矩阵表示时，该矩阵至少有()个非零元素。

- A. n B. 2(n-1) C. n/2 D. n²

答案: B

(5) G是一个非连通无向图，共有28条边，则该图至少有()个顶点。

- A. 7 B. 8 C. 9 D. 10

答案: C

解释: 8个顶点的无向图最多有 $8*7/2=28$ 条边，再添加一个点即构成非连通无向图，故至少有9个顶点。

(6) 若从无向图的任意一个顶点出发进行一次深度优先搜索可以访问图中所有的顶点，则该图一定是()图。

- A. 非连通 B. 连通 C. 强连通 D. 有向

答案: B

解释: 即从该无向图任意一个顶点出发有到各个顶点的路径，所以该无向图是连通图。

(7) 下面()算法适合构造一个稠密图G的最小生成树。

- A. Prim 算法 B. Kruskal 算法 C. Floyd 算法 D. Dijkstra 算法

答案：B

2. 应用题

(1) 已知图 6.32 所示的有向图，请给出：

- ① 每个顶点的入度和出度； ② 邻接矩阵； ③ 邻接表； ④ 逆邻接表。

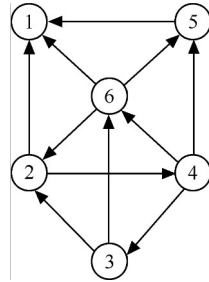


图 6.32 有向图

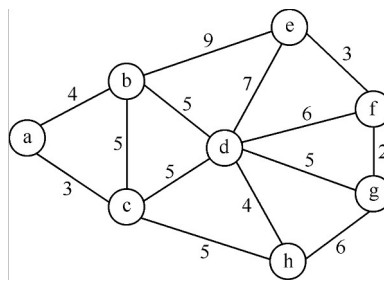


图 6.33 无向网

答案：

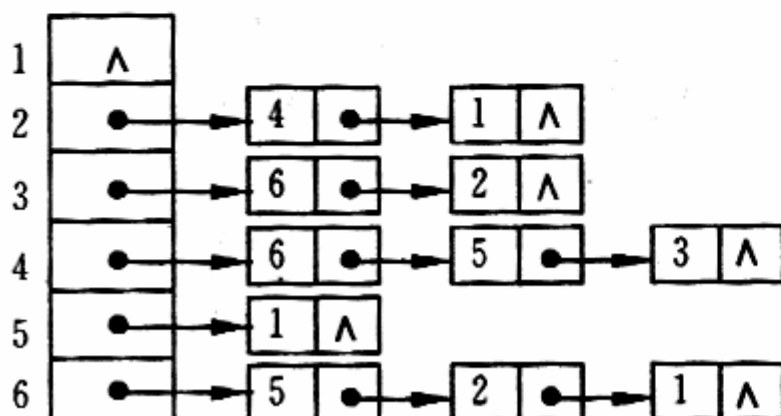
(1)

顶点	1	2	3	4	5	6
入度	3	2	1	1	2	2
出度	0	2	2	3	1	3

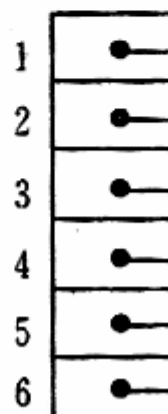
(2) 邻接矩阵

0	0
1	0
0	1
0	0
1	0
1	1

(3) 邻接表



(4) 逆邻接表



(2) 已知如图 6.33 所示的无向网，请给出：

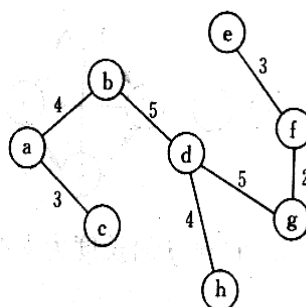
- ① 邻接矩阵；
- ② 邻接表；
- ③ 最小生成树

答案：

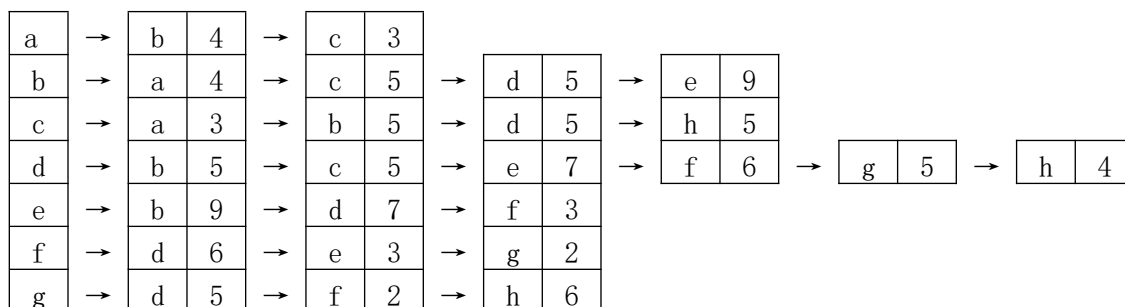
①

∞	4	3	∞	∞	∞	∞	∞
4	∞	5	5	9	∞	∞	∞
3	5	∞	5	∞	∞	∞	5
∞	5	5	∞	7	6	5	4
∞	9	∞	7	∞	3	∞	∞
∞	∞	∞	6	3	∞	2	∞
∞	∞	∞	5	∞	2	∞	6
∞	∞	5	4	∞	∞	6	∞

③

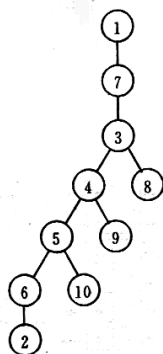


②

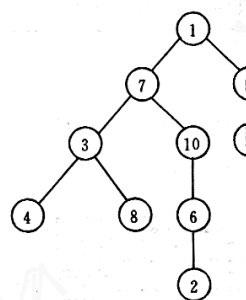


(3) 已知图的邻接矩阵如图 6.34 所示。试分别画出自顶点 1 出发进行遍历所得的深度优先生成树和广度优先生成树。

深度优先生成树



广度优先生成树



(4) 有向网如图 6.35 所示，试用迪杰算法求出从顶点 a 到其他各顶点间的最完成表 6.9。

	1	2	3	4	5	6	7	8	9	10
1	0	0	0	0	0	0	1	0	1	0
2	0	0	1	0	0	0	1	0	0	0
3	0	0	0	1	0	0	0	1	0	0
4	0	0	0	0	1	0	0	0	1	0
5	0	0	0	0	0	1	0	0	0	1
6	1	1	0	0	0	0	0	0	0	0
7	0	0	1	0	0	0	0	0	0	1
8	1	0	0	1	0	0	0	0	1	0
9	0	0	0	0	1	0	1	0	0	1
10	1	0	0	0	0	1	0	0	0	0

图 6.28 邻接矩阵

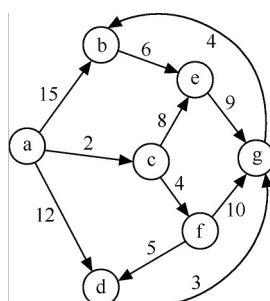


图 6.35 有向网

斯特拉
最短路径，

表 6.9

D	i=1	i=2	i=3	i=4	i=5	i=6
终点						

b	15 (a,b)	15 (a,b)	15 (a,b)	15 (a,b)	15 (a,b)	<u>15</u> <u>(a,b)</u>
c	<u>2</u> <u>(a,c)</u>					
d	12 (a,d)	12 (a,d)	11 (a,c,f,d)	<u>11</u> <u>(a,c,f,d)</u>		
e	∞	10 (a,c,e)	<u>10</u> <u>(a,c,e)</u>			
f	∞	<u>6</u> <u>(a,c,f)</u>				
g	∞	∞	16 (a,c,f,g)	16 (a,c,f,g)	<u>14</u> <u>(a,c,f,d,g)</u>	
S 终 点 集	{a,c}	{a,c,f}	{a,c,f,e}	{a,c,f,e,d}	{a,c,f,e,d,g}	{a,c,f,e,d,g},b}

(5) 试对图 6.36 所示的 AOE-网:

- ① 求这个工程最早可能在什么时间结束;
- ② 求每个活动的最早开始时间和最开始时间;
- ③ 确定哪些活动是关键活动

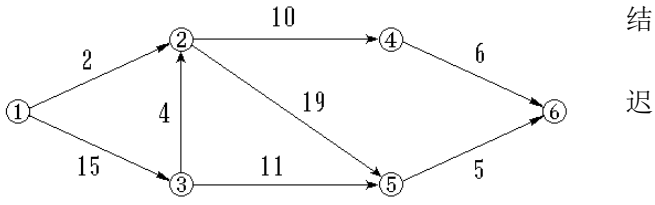


图 6.36 AOE-网

答案: 按拓扑有序的顺序计算各个顶点的最早可能开始时间 V_e 和最迟允许开始时间 V_l 。然后再计算各个活动的最早可能开始时间 e 和最迟允许开始时间 l ，根据 $l - e = 0$ 来确定关键活动，从而确定关键路径。

	1 ∂	2 $\div$	3 $\bullet$	4 $\neq$	5 $\equiv$	6 $\approx$
V_e	0	19	15	29	38	43
V_l	0	19	15	37	38	43

	<1, 2>	<1, 3>	<3, 2>	<2, 4>	<2, 5>	<3, 5>	<4, 6>	<5, 6>
e	0	0	15	19	19	15	29	38
l	17	0	15	27	19	27	37	38
$-e$	17	0	0	8	0	12	8	0

此工程最早完成时间为 43。关键路径为<1, 3><3, 2><2, 5><5, 6>

第 7 章 查找

1. 选择题

(1) 对 n 个元素的表做顺序查找时，若查找每个元素的概率相同，则平均查找长度为 ()。

- A. $(n-1)/2$ B. $n/2$ C. $(n+1)/2$ D. n

答案: C

解释: 总查找次数 $N=1+2+3+\cdots+n=n(n+1)/2$ ，则平均查找长度为 $N/n=(n+1)/2$ 。

(2) 适用于折半查找的表的存储方式及元素排列要求为 ()。

- A. 链接方式存储，元素无序 B. 链接方式存储，元素有序
C. 顺序方式存储，元素无序 D. 顺序方式存储，元素有序

答案: D

解释: 折半查找要求线性表必须采用顺序存储结构，而且表中元素按关键字有序排列。

(3) 如果要求一个线性表既能较快的查找，又能适应动态变化的要求，最好采用 () 查找法。

- A. 顺序查找 B. 折半查找
C. 分块查找 D. 哈希查找

答案: C

解释: 分块查找的优点是: 在表中插入和删除数据元素时，只要找到该元素对应的块，就可以在该块内进行插入和删除运算。由于块内是无序的，故插入和删除比较容易，无需进行大量移动。如果线性表既要快速查找又经常动态变化，则可采用分块查找。

(4) 折半查找有序表 (4, 6, 10, 12, 20, 30, 50, 70, 88, 100)。若查找表中元素 58，则它将依次与表中 () 比较大小，查找结果是失败。

- A. 20, 70, 30, 50 B. 30, 88, 70, 50
C. 20, 50 D. 30, 88, 50

答案: A

解释: 表中共 10 个元素，第一次取 $\lfloor (1+10)/2 \rfloor = 5$ ，与第五个元素 20 比较，58 大于 20，再取 $\lfloor (6+10)/2 \rfloor = 8$ ，与第八个元素 70 比较，依次类推再与 30、50 比较，最终查找失败。

(5) 对 22 个记录的有序表作折半查找，当查找失败时，至少需要比较 () 次关键字。

- A. 3 B. 4 C. 5 D. 6

答案: B

解释: 22 个记录的有序表，其折半查找的判定树深度为 $\lfloor \log_2 22 \rfloor + 1 = 5$ ，且该判定树不是满二叉树，即查找失败时至多比较 5 次，至少比较 4 次。

(6) 折半搜索与二叉排序树的时间性能 ()。

- A. 相同 B. 完全不同
C. 有时不相同 D. 数量级都是 $O(\log_2 n)$

答案: C

(7) 分别以下列序列构造二叉排序树，与用其它三个序列所构造的结果不同的是 ()。

- A. (100, 80, 90, 60, 120, 110, 130)
B. (100, 120, 110, 130, 80, 60, 90)
C. (100, 60, 80, 90, 120, 110, 130)
D. (100, 80, 60, 90, 120, 130, 110)

答案: C

解释: A、B、C、D 四个选项构造二叉排序树都以 100 为根，易知 A、B、D 三个序列中第一个比 100 小的关键字为 80，即 100 的左孩子为 80，而 C 选项中 100 的左孩子为 60，故选 C。

(8) 在平衡二叉树中插入一个结点后造成了不平衡，设最低的不平衡结点为 A，并已知 A 的左孩子的平衡因子为 0 右孩子的平衡因子为 1，则应作 () 型调整以使其平衡。

- A. LL B. LR C. RL D. RR

答案: C

(9) 下列关于 m 阶 B-树 的说法错误的是 ()。

- A. 根结点至多有 m 棵子树
- B. 所有叶子都在同一层次上
- C. 非叶结点至少有 $m/2$ (m 为偶数) 或 $m/2+1$ (m 为奇数) 棵子树
- D. 根结点中的数据是有序的

答案: D

(10) 下面关于 B-树 和 B+树 的叙述中, 不正确的是 ()。

- A. B-树 和 B+树 都是平衡的多叉树
- B. B-树 和 B+树 都可用于文件的索引结构
- C. B-树 和 B+树 都能有效地支持顺序检索
- D. B-树 和 B+树 都能有效地支持随机检索

答案: C

(11) m 阶 B-树 是一棵 ()。

- A. m 叉排序树
- B. m 叉平衡排序树
- C. $m-1$ 叉平衡排序树
- D. $m+1$ 叉平衡排序树

答案: B

(12) 下面关于哈希查找的说法, 正确的是 ()。

- A. 哈希函数构造的越复杂越好, 因为这样随机性好, 冲突小
- B. 除留余数法是所有哈希函数中最好的
- C. 不存在特别好与坏的哈希函数, 要视情况而定
- D. 哈希表的平均查找长度有时也和记录总数有关

答案: C

(13) 下面关于哈希查找的说法, 不正确的是 ()。

- A. 采用链地址法处理冲突时, 查找一个元素的时间是相同的
- B. 采用链地址法处理冲突时, 若插入规定总是在链首, 则插入任一个元素的时间是相同的
- C. 用链地址法处理冲突, 不会引起二次聚集现象
- D. 用链地址法处理冲突, 适合表长不确定的情况

答案: A

解释: 在同义词构成的单链表中, 查找该单链表表中不同元素, 所消耗的时间不同。

(14) 设哈希表长为 14, 哈希函数是 $H(key)=key\%11$, 表中已有数据的关键字为 15, 38, 61, 84 共四个, 现要将关键字为 49 的元素加到表中, 用二次探测法解决冲突, 则放入的位置是 ()。

- A. 8
- B. 3
- C. 5
- D. 9

答案: D

解释: 关键字 15 放入位置 4, 关键字 38 放入位置 5, 关键字 61 放入位置 6, 关键字 84 放入位置 7, 再添加关键字 49, 计算得到地址为 5, 冲突, 用二次探测法解决冲突得到新地址为 6, 仍冲突, 再用二次探测法解决冲突, 得到新地址为 4, 仍冲突, 再用二次探测法解决冲突, 得到新地址为 9, 不冲突, 即将关键字 49 放入位置 9。

(15) 采用线性探测法处理冲突, 可能要探测多个位置, 在查找成功的情况下, 所探测的这些位置上的关键字 ()。

- A. 不一定是同义词
- B. 一定都是同义词
- C. 一定都不是同义词
- D. 都相同

答案: A

解释: 所探测的这些关键字可能是在处理其它关键字冲突过程中放入该位置的。

2. 应用题

(1) 假定对有序表: (3, 4, 5, 7, 24, 30, 42, 54, 63, 72, 87, 95) 进行折半查找, 试回答下列问题:

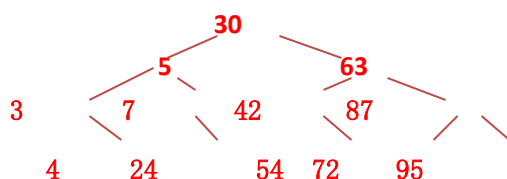
- ① 画出描述折半查找过程的判定树;
- ② 若查找元素 54, 需依次与哪些元素比较?

③ 若查找元素 90，需依次与哪些元素比较？

④ 假定每个元素的查找概率相等，求查找成功时的平均查找长度。

答案：

①先画出判定树如下（注： $\text{mid}=\lfloor(1+12)/2\rfloor=6$ ）：



②查找元素 54，需依次与 30, 63, 42, 54 元素比较；

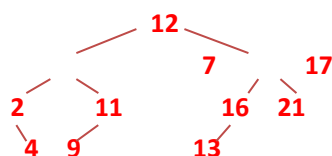
③查找元素 90，需依次与 30, 63, 87, 95 元素比较；

④求 ASL 之前，需要统计每个元素的查找次数。判定树的前 3 层共查找 $1+2\times 2+4\times 3=17$ 次；但最后一层未满，不能用 8×4 ，只能用 $5\times 4=20$ 次，

所以 $\text{ASL}=1/12(17+20)=37/12\approx 3.08$

(2) 在一棵空的二叉排序树中依次插入关键字序列为 12, 7, 17, 11, 16, 2, 13, 9, 21, 4, 请画出所得到的二叉排序树。

答案：



验算方法： 用中序遍历应得到排序结果：2,4,7,9,11,12,13,16,17,21

(3) 已知如下所示长度为 12 的表：（Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec）

① 试按表中元素的顺序依次插入一棵初始为空的二叉排序树，画出插入完成之后的二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。

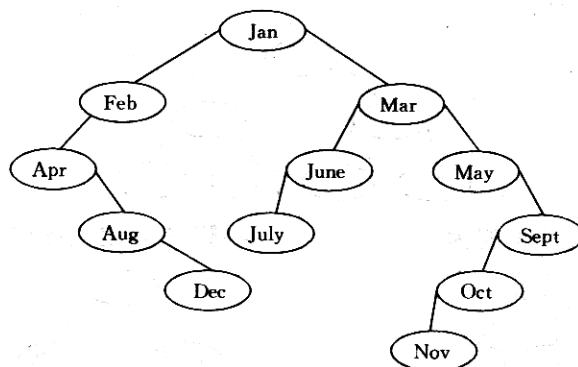
② 若对表中元素先进行排序构成有序表，求在等概率的情况下对此有序表进行折半查找时查找成功的平均查找长度。

③ 按表中元素顺序构造一棵平衡二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。

答案：

(1) 求得的二叉排序树如下图所示，在等概率情况下查找成功的平均查找长度为

$$\text{ASL}_{\text{succ}} = \frac{1}{12}(1 \times 1 + 2 \times 2 + 3 \times 3 + 4 \times 3 + 5 \times 2 + 6 \times 1) = \frac{42}{12}$$



(2) 经排序后的表及在折半查找时找到表中元

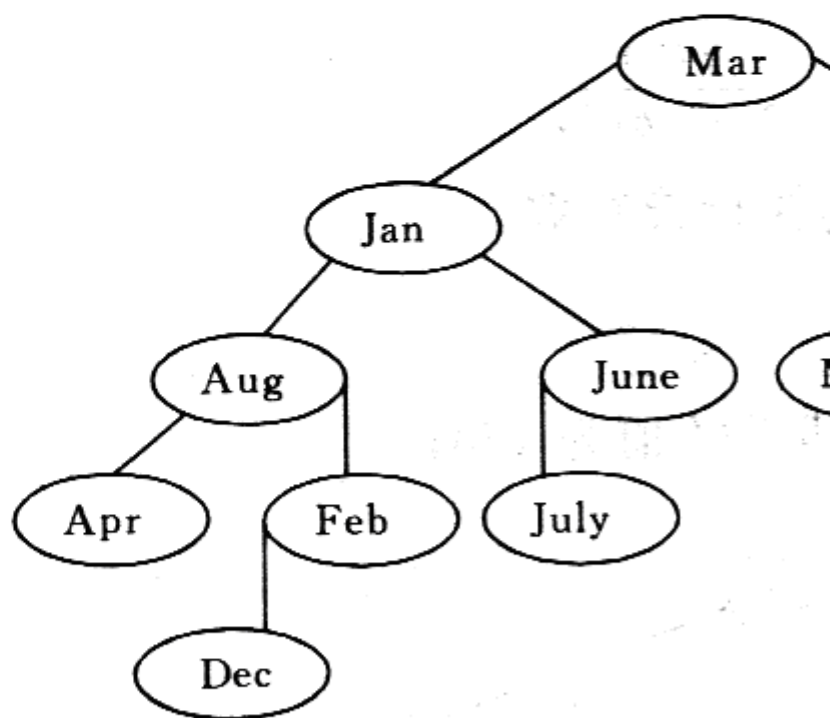
Apr	Aug	Dec	Feb	Jan	July	June
3	4	2	3	4	1	3

等概率情况下查找成功时的平均查找长度为

$$ASL_{succ} = \frac{1}{12}(1 \times 1 + 2 \times 2 + 3 \times 3)$$

(3)

平衡二叉树



它在等概率情况下的平均查找长度为

$$ASL = \frac{1}{12}(1 \times 1 + 2 \times 2 + 3 \times 4)$$

(4) 对图 7.31 所示的 3 阶 B-树，依次执行下列操作，画出各步操作的结果。

- ① 插入 90 ② 插入 25 ③ 插入 45 ④ 删除 60

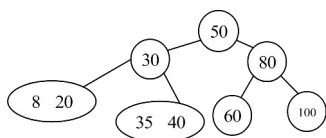
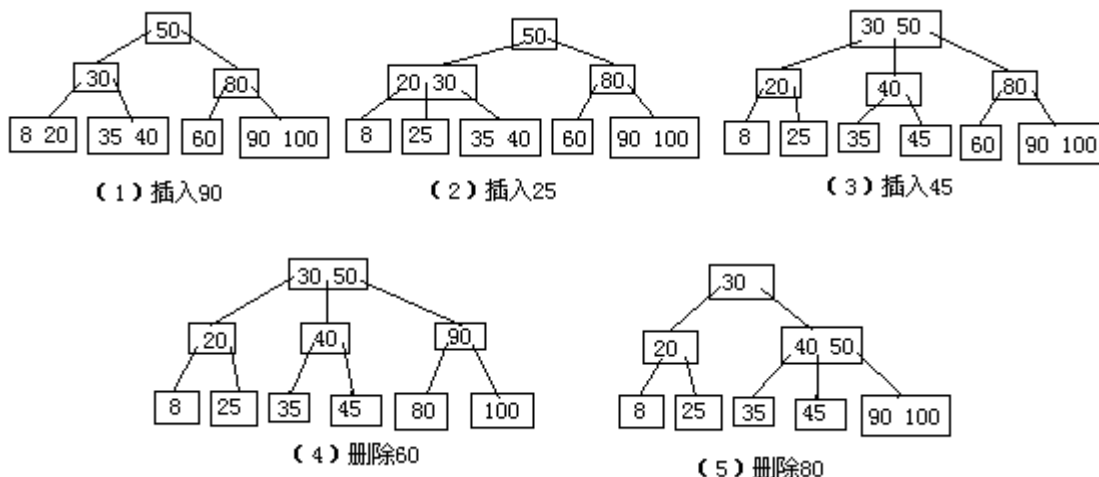


图 7.31 3 阶 B-树



(5) 设哈希表的地址范围为 0~17，哈希函数为： $H(\text{key})=\text{key}\%16$ 。用线性探测法处理冲突，输入关键字序列：(10, 24, 32, 17, 31, 30, 46, 47, 40, 63, 49)，构造哈希表，试回答下列问题：

- ① 画出哈希表的示意图；
- ② 若查找关键字 63，需要依次与哪些关键字进行比较？
- ③ 若查找关键字 60，需要依次与哪些关键字比较？
- ④ 假定每个关键字的查找概率相等，求查找成功时的平均查找长度。

答案：

①画表如下：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
32	17	63	49					24	40	10				30	31	46	47

②查找 63，首先要与 $H(63)=63\%16=15$ 号单元内容比较，即 63 与 31 比较，不匹配；然后顺移，与 46, 47, 32, 17, 63 相比，一共比较了 6 次！

③查找 60，首先要与 $H(60)=60\%16=12$ 号单元内容比较，但因为 12 号单元为空（应当有空标记），所以应当只比较这一次即可。

④对于黑色数据元素，各比较 1 次；共 6 次；

对红色元素则各不相同，要统计移位的位数。“63”需要 6 次，“49”需要 3 次，“40”需要 2 次，“46”需要 3 次，“47”需要 3 次，

所以 $ASL=1/11 (6+2+3\times3+6) =23/11$

(6) 设有一组关键字(9, 01, 23, 14, 55, 20, 84, 27)，采用哈希函数： $H(\text{key})=\text{key} \%7$ ，表长为 10，用开放地址法的二次探测法处理冲突。要求：对该关键字序列构造哈希表，并计算查找成功的平均查找长度。

答案：

散列地址	0	1	2	3	4	5	6	7	8	9
关键字	14	1	9	23	84	27	55	20		
比较次数	1	1	1	2	3	4	1	2		

平均查找长度： $ASL_{succ} = (1+1+1+2+3+4+1+2) / 8 = 15/8$

以关键字 27 为例： $H(27) = 27\%7 = 6$ （冲突） $H_1 = (6+1) \% 10 = 7$ （冲突）

$H_2 = (6+2^2) \% 10 = 0$ （冲突） $H_3 = (6+3^3) \% 10 = 5$ 所以比较了 4 次。

(7) 设哈希函数 $H(K) = 3K \bmod 11$, 哈希地址空间为 $0 \sim 10$, 对关键字序列 (32, 13, 49, 24, 38, 21, 4, 12), 按下述两种解决冲突的方法构造哈希表, 并分别求出等概率下查找成功时和查找失败时的平均查找长度 ASL_{succ} 和 ASL_{unsucc} 。

① 线性探测法;

② 链地址法。

答案:

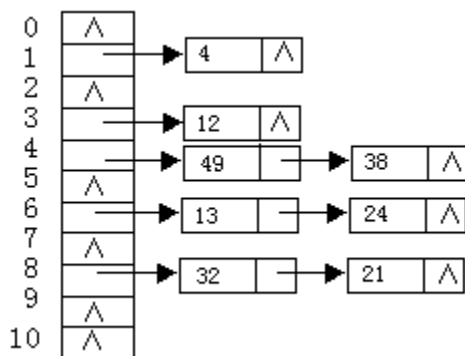
①

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字		4		12	49	38	13	24	32	21	
比较次数		1		1	1	2	1	2	1	2	

$ASL_{succ} = (1+1+1+2+1+2+1+2) / 8 = 11/8$

$ASL_{unsucc} = (1+2+1+8+7+6+5+4+3+2+1) / 11 = 40/11$

②



$ASL_{succ} = (1*5+2*3) / 8 = 11/8$

$ASL_{unsucc} = (1+2+1+2+3+1+3+1+3+1+1) / 11 = 19/11$

第 8 章 排序

1. 选择题

(1) 从未排序序列中依次取出元素与已排序序列中的元素进行比较, 将其放入已排序序列的正确位置上的方法, 这种排序方法称为 ()。

- A. 归并排序 B. 冒泡排序 C. 插入排序 D. 选择排序

答案: C

(2) 从未排序序列中挑选元素, 并将其依次放入已排序序列 (初始时空) 的一端的方法, 称为 ()。

- A. 归并排序 B. 冒泡排序 C. 插入排序 D. 选择排序

答案: D

(3) 对 n 个不同的关键字由小到大进行冒泡排序, 在下列 () 情况下比较的次数最多。

- A. 从小到大排列好的 B. 从大到小排列好的

C. 元素无序 D. 元素基本有序

答案: B

解释: 对关键字进行冒泡排序, 关键字逆序时比较次数最多。

(4) 对 n 个不同的排序码进行冒泡排序, 在元素无序的情况下比较的次数最多为 ()。

A. $n+1$ B. n C. $n-1$ D. $n(n-1)/2$

答案: D

解释: 比较次数最多时, 第一次比较 $n-1$ 次, 第二次比较 $n-2$ 次……最后一次比较 1 次, 即 $(n-1)+(n-2)+\cdots+1=n(n-1)/2$ 。

(5) 快速排序在下列 () 情况下最易发挥其长处。

A. 被排序的数据中含有多个相同排序码
B. 被排序的数据已基本有序
C. 被排序的数据完全无序
D. 被排序的数据中的最大值和最小值相差悬殊

答案: C

解释: B 选项是快速排序的最坏情况。

(6) 对 n 个关键字作快速排序, 在最坏情况下, 算法的时间复杂度是 ()。

A. $O(n)$ B. $O(n^2)$ C. $O(n\log_2 n)$ D. $O(n^3)$

答案: B

解释: 快速排序的平均时间复杂度为 $O(n\log_2 n)$, 但在最坏情况下, 即关键字基本排好序的情况下, 时间复杂度为 $O(n^2)$ 。

(7) 若一组记录的排序码为 (46, 79, 56, 38, 40, 84), 则利用快速排序的方法, 以第一个记录为基准得到的一次划分结果为 ()。

A. 38, 40, 46, 56, 79, 84 B. 40, 38, 46, 79, 56, 84
C. 40, 38, 46, 56, 79, 84 D. 40, 38, 46, 84, 56, 79

答案: C

(8) 下列关键字序列中, () 是堆。

A. 16, 72, 31, 23, 94, 53 B. 94, 23, 31, 72, 16, 53
C. 16, 53, 23, 94, 31, 72 D. 16, 23, 53, 31, 94, 72

答案: D

解释: D 选项为小根堆

(9) 堆是一种 () 排序。

A. 插入 B. 选择 C. 交换 D. 归并

答案: B

(10) 堆的形状是一棵 ()。

A. 二叉排序树 B. 满二叉树 C. 完全二叉树 D. 平衡二叉树

答案: C

(11) 若一组记录的排序码为 (46, 79, 56, 38, 40, 84), 则利用堆排序的方法建立的初始堆为 ()。

A. 79, 46, 56, 38, 40, 84 B. 84, 79, 56, 38, 40, 46
C. 84, 79, 56, 46, 40, 38 D. 84, 56, 79, 40, 46, 38

答案: B

(12) 下述几种排序方法中, 要求内存最大的是 ()。

A. 希尔排序 B. 快速排序 C. 归并排序 D. 堆排序

答案: C

解释: 堆排序、希尔排序的空间复杂度为 $O(1)$, 快速排序的空间复杂度为 $O(\log_2 n)$, 归并排序的空间复杂度为 $O(n)$ 。

(13) 下述几种排序方法中, () 是稳定的排序方法。

- A. 希尔排序 B. 快速排序 C. 归并排序 D. 堆排序

答案: C

解释: 不稳定排序有希尔排序、简单选择排序、快速排序、堆排序; 稳定排序有直接插入排序、折半插入排序、冒泡排序、归并排序、基数排序。

(14) 数据表中有 10000 个元素, 如果仅要求求出其中最大的 10 个元素, 则采用 () 算法最节省时间。

- A. 冒泡排序 B. 快速排序 C. 简单选择排序 D. 堆排序

答案: D

(15) 下列排序算法中, () 不能保证每趟排序至少能将一个元素放到其最终的位置上。

- A. 希尔排序 B. 快速排序 C. 冒泡排序 D. 堆排序

答案: A

解释: 快速排序的每趟排序能将作为枢轴的元素放到最终位置; 冒泡排序的每趟排序能将最大或最小的元素放到最终位置; 堆排序的每趟排序能将最大或最小的元素放到最终位置。

2. 应用题

(1) 设待排序的关键字序列为{12, 2, 16, 30, 28, 10, 16*, 20, 6, 18}, 试分别写出使用以下排序方法, 每趟排序结束后关键字序列的状态。

- ① 直接插入排序 ② 折半插入排序 ③ 希尔排序 (增量选取 5, 3, 1) ④ 冒泡排序
⑤ 快速排序 ⑥ 简单选择排序 ⑦ 堆排序 ⑧ 二路归并排序

答案:

① 直接插入排序

```
[2 12] 16 30 28 10 16* 20 6 18
[2 12 16] 30 28 10 16* 20 6 18
[2 12 16 30] 28 10 16* 20 6 18
[2 12 16 28 30] 10 16* 20 6 18
[2 10 12 16 28 30] 16* 20 6 18
[2 10 12 16 16* 28 30] 20 6 18
[2 10 12 16 16* 20 28 30] 6 18
[2 6 10 12 16 16* 20 28 30] 18
[2 6 10 12 16 16* 18 20 28 30]
```

② 折半插入排序 排序过程同①

③ 希尔排序 (增量选取 5, 3, 1)

```
10 2 16 6 18 12 16* 20 30 28 (增量选取 5)
6 2 12 10 18 16 16* 20 30 28 (增量选取 3)
2 6 10 12 16 16* 18 20 28 30 (增量选取 1)
```

④ 冒泡排序

```
2 12 16 28 10 16* 20 6 18 [30]
2 12 16 10 16* 20 6 18 [28 30]
2 12 10 16 16* 6 18 [20 28 30]
2 10 12 16 6 16* [18 20 28 30]
2 10 12 6 16 [16* 18 20 28 30]
2 10 6 12 [16 16* 18 20 28 30]
2 6 10 [12 16 16* 18 20 28 30]
```

2 6 10 12 16 16* 18 20 28 30]

⑤ 快速排序

12 [6 2 10] **12** [28 30 16* 20 16 18]

6 [2] **6** [10] 12 [28 30 16* 20 16 18]

28 2 6 10 12 [18 16 16* 20] **28** [30]

18 2 6 10 12 [16* 16] **18** [20] 28 30

16* 2 6 10 12 16* [16] 18 20 28 30

左子序列递归深度为1，右子序列递归深度为3

⑥ 简单选择排序

2 [12 16 30 28 10 16* 20 6 18]
 2 6 [16 30 28 10 16* 20 12 18]
 2 6 10 [30 28 16 16* 20 12 18]
 2 6 10 12 [28 16 16* 20 30 18]
 2 6 10 12 16 [28 16* 20 30 18]
 2 6 10 12 16 16* [28 20 30 18]
 2 6 10 12 16 16* 18 [20 30 28]
 2 6 10 12 16 16* 18 20 [28 30]
 2 6 10 12 16 16* 18 20 28 [30]

⑧ 二路归并排序

2 12 16 30 10 28 16* 20 6 18
2 12 16 30 10 16* 20 28 6 18
2 10 12 16 16* 20 28 30 6 18
2 6 10 12 16 16* 18 20 28 30

(2) 给出如下关键字序列{321, 156, 57, 46, 28, 7, 331, 33, 34, 63}，试按链式基数排序方法，列出每一趟分配和收集的过程。

答案：

按最低位优先法 → 321 → 156 → 57 → 46 → 28 → 7 → 331 → 33 → 34 → 63

分配 [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

321 33 34 156 57 28

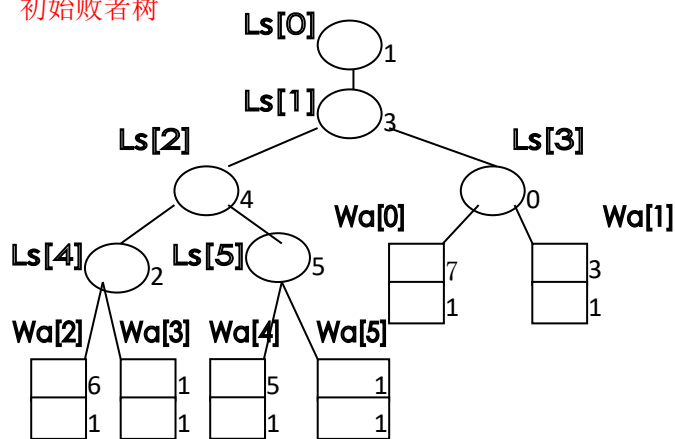
331 63 46 7

收集 → 321 → 331 → 33 → 63 → 34 → 156 → 46 → 57 → 7 → 28

(3) 对输入文件(101, 51, 19, 61, 3, 71, 31, 17, 19, 100, 55, 20, 9, 30, 50, 6, 90)；当k=6时，使用置换-选择算法，写出建立的初始败者树及生成的初始归并段。

答案：

初始败者树



初始归并段: $R_1: 3, 19, 31, 51, 61, 71, 100, 101$

$R_2: 9, 17, 19, 20, 30, 50, 55, 90$

$R_3: 6$